

Московский государственный технический
университет им. Н. Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”

Курс “Парадигмы и конструкции языков программирования”
Отчёт по рубежному контролю №2

Выполнил:
Студент группы ИУ5-31Б
Луценко Александра

Проверил:
Гапанюк Ю.Е

refactoredcode.py

```
class Computer:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class Browser:
    def __init__(self, id, name, memory_usage, computer_id):
        self.id = id
        self.name = name
        self.memory_usage = memory_usage
        self.computer_id = computer_id

class BrowserComputer:
    def __init__(self, browser_id, computer_id):
        self.browser_id = browser_id
        self.computer_id = computer_id

computers = [
    Computer(1, "Игровой компьютер"),
    Computer(2, "Рабочий компьютер"),
    Computer(3, "Серверный отдел"),
    Computer(4, "Ноутбук отдела разработки"),
    Computer(5, "Компьютер отдела тестирования")
]

browsers = [
    Browser(1, "Chrome", 512, 1),
    Browser(2, "Firefox", 256, 2),
    Browser(3, "Edge", 300, 3),
    Browser(4, "Opera", 200, 4),
    Browser(5, "Safari", 350, 5),
    Browser(6, "Chrome", 500, 2),
    Browser(7, "Firefox", 280, 1),
    Browser(8, "Edge", 320, 4)
]

browser_computers = [
    BrowserComputer(1, 1), BrowserComputer(1, 2),
    BrowserComputer(2, 2), BrowserComputer(2, 4),
    BrowserComputer(3, 3), BrowserComputer(4, 4),
    BrowserComputer(5, 5), BrowserComputer(6, 1),
    BrowserComputer(6, 2), BrowserComputer(7, 2),
```

```

    BrowserComputer(7, 5), BrowserComputer(8, 3),
    BrowserComputer(8, 4)
]

def get_computer_browsers_one_to_many(computers, browsers):
    return [(comp, [br for br in browsers if br.computer_id == comp.id])
            for comp in computers]

def get_browser_computers_many_to_many(computers, browsers,
browser_computers):
    result = []
    for br in browsers:
        computer_ids = [bc.computer_id for bc in browser_computers if bc.browser_id
== br.id]
        comps = [comp for comp in computers if comp.id in computer_ids]
        result.append((br, comps))
    return result

def get_computers_with_total_memory(computers, browsers):
    comp_totals = [(comp, sum(br.memory_usage for br in browsers_list))
                    for comp, browsers_list in
get_computer_browsers_one_to_many(computers, browsers)]
    return sorted(comp_totals, key=lambda x: x[1], reverse=True)

def get_computers_with_department(computers, browsers, keyword="отдел"):
    result = []
    for comp, browsers_list in get_computer_browsers_one_to_many(computers,
browsers):
        if keyword in comp.name.lower():
            browser_names = [br.name for br in browsers_list]
            result.append((comp, browser_names))
    return result

def variant_a(computers, browsers, browser_computers):
    print("=== ВАРИАНТ А ===")
    print("Предметная область: Браузер-Компьютер\n")

    print("1. Список связанных компьютеров и браузеров:")
    computer_browsers = sorted(get_computer_browsers_one_to_many(computers,
browsers), key=lambda x: x[0].name)
    for comp, browsers_list in computer_browsers:
        browser_names = [br.name for br in browsers_list]
        print(f'{comp.name}: {browser_names}')

    print("\n2. Компьютеры с суммарным использованием памяти браузеров:")

```

```

comp_totals = get_computers_with_total_memory(computers, browsers)
for comp, total in comp_totals:
    print(f'{comp.name}: {total} МБ")

print("\n3. Компьютеры с 'отдел' в названии и их браузеры:")
dept_computers = get_computers_with_department(computers, browsers)
for comp, browser_names in dept_computers:
    print(f'{comp.name}: {browser_names}")

if __name__ == "__main__":
    variant_a(computers, browsers, browser_computers)

```

test_code.py

```

import unittest
from refactored_code import (
    Computer, Browser, BrowserComputer,
    get_computer_browsers_one_to_many,
    get_computers_with_total_memory,
    get_computers_with_department
)

class TestBrowserComputerLogic(unittest.TestCase):

    def setUp(self):
        self.computers = [
            Computer(1, "Игровой компьютер"),
            Computer(2, "Рабочий компьютер"),
            Computer(3, "Серверный отдел")
        ]
        self.browsers = [
            Browser(1, "Chrome", 512, 1),
            Browser(2, "Firefox", 256, 2),
            Browser(3, "Edge", 300, 3),
            Browser(4, "Chrome", 500, 2)
        ]

    def test_get_computer_browsers_one_to_many(self):
        result = get_computer_browsers_one_to_many(self.computers, self.browsers)
        self.assertEqual(len(result), 3)
        computer_names = [comp.name for comp, _ in result]
        self.assertIn("Игровой компьютер", computer_names)
        self.assertIn("Рабочий компьютер", computer_names)

```

```
self.assertIn("Серверный отдел", computer_names)

def test_get_computers_with_total_memory(self):
    result = get_computers_with_total_memory(self.computers, self.browsers)
    self.assertEqual(len(result), 3)
    memory_values = [total for _, total in result]
    self.assertEqual(memory_values, sorted(memory_values, reverse=True))

def test_get_computers_with_department(self):
    result = get_computers_with_department(self.computers, self.browsers)
    self.assertEqual(len(result), 1)
    comp_name, browsers = result[0]
    self.assertEqual(comp_name.name, "Серверный отдел")
    self.assertEqual(browsers, ["Edge"])

if __name__ == "__main__":
    unittest.main()
```

```
...
-----
Ran 3 tests in 0.001s

OK
```